DATA SCIENCE LAB EXAM STUDY GUIDE

Program 06: Gaussian Naive Bayes Classifier

Source File: `e06_naive_bayes.py` | College Practical Exam Preparation

Table of Contents

Section 1: Program Identification and Aim — Page 1

Section 2: Prerequisites — Page 1

Section 3: Complete Exam-Ready Algorithm — Page 2

Section 4: Complete Theory (Bayes' Theorem, Normal PDF, Precision/Recall/F1) — Page 3

Section 5: Complete Original Source Code & Block Walkthrough — Page 5

Section 6: Complete Function, Parameter and Syntax Reference — Page 6

Section 7: Step-by-Step Execution Walkthrough — Page 7

Section 8: Output and Interpretation — Page 8

Section 9: Exam Preparation (Record Answer, Question Bank, Model Answers) — Page 9

Page 10

C. Predict-the-Output Questions

1. **Question:** In `e06_naive_bayes.py`, what are the three class names in the classification report?
   Answer: `setosa`, `versicolor`, and `virginica`.
2. **Question:** With `test_size=0.3, random_state=40`, how many samples are in the test set?
   Answer: Exactly 45 samples ($150 imes 0.3 = 45$).

D. Identify-the-Function Questions

1. **Question:** Which scikit-learn class implements Gaussian Naive Bayes for continuous data?
   Answer: `GaussianNB()` from `sklearn.naive_bayes`.
2. **Question:** Which function generates a text report showing Precision, Recall, F1-Score, and Support?
   Answer: `classification_report()` from `sklearn.metrics`.
3. **Question:** Which function loads the benchmark Iris dataset?
   Answer: `load_iris()` from `sklearn.datasets`.

Section 10: Viva Voce Questions and Answers (16 Detailed Q&As)

Section 11: Common Mistakes and Troubleshooting — Page 12

Section 12: Memory Tricks and Last-Minute Rapid Revision — Page 13

Section 13: Cross-Program Comparison Table (Programs 5 to 10) — Page 14

Section 1: Program Identification and Aim

- **Program Number:** Program 06 (Machine Learning Series 2 of 6)
- **Exact Filename:** `e06_naive_bayes.py`
- **Actual Question from Source:** "Question 1: Program to implement Naive Bayes Algorithm using any standard dataset available in the public domain and find the accuracy of the algorithm."
- **Aim of the Program:** To implement the Gaussian Naive Bayes probabilistic classification algorithm on the standard Iris flower dataset, train the model using a fixed random seed (`random_state=40`), evaluate overall classification accuracy, and print a comprehensive multi-class classification report detailing precision, recall, and F1-score for each species.
- **Problem Statement & Objectives:**
  1. Load the 150 Iris flower samples containing 4 continuous measurements.
  2. Split the dataset into 70% training (105 samples) and 30% testing (45 samples) using `random_state=40` for reproducible benchmark results.
  3. Instantiate and train a `GaussianNB` classifier.
  4. Predict classes for the 45 test samples.
  5. Compute and print the exact test accuracy score and the full classification report mapped to `iris.target_names`.
- **Input Data:** Scikit-Learn's built-in `load_iris()` dataset.
- **Target / Output:** Integer class labels ($0, 1, 2$) corresponding to Setosa, Versicolor, and Virginica.
- **Expected Results:** Deterministic 1.0 (100%) accuracy and a complete classification report table.

Section 2: Prerequisites

To master this program, understand these core probabilistic concepts:

1. **Conditional Probability:** The probability of event A occurring given that event B has already occurred, denoted as $P(A|B)$.
2. **Bayes' Theorem:** Mathematical formula for updating prior probabilities when presented with new observed evidence: $$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$$

3. **Conditional Independence:** Assuming that the presence of one feature does not affect the probability of another feature given the class label ($P(x_1, x_2|y) = P(x_1|y) \cdot P(x_2|y)$).
4. **Gaussian (Normal) Distribution:** A symmetric, bell-shaped probability density function characterized by its mean (μ) and variance (σ^2).
5. **Classification Report Metrics:**
    - **Precision:** Out of all samples predicted as Class C, how many were truly Class C? ($\frac{TP}{TP + FP}$).
    - **Recall (Sensitivity):** Out of all actual Class C samples, how many did the model identify? ($\frac{TP}{TP + FN}$).
    - **F1-Score:** Harmonic mean of Precision and Recall ($2 \times \frac{P \times R}{P + R}$).
    - **Support:** Number of actual ground-truth occurrences of that class in the test dataset.

Section 3: Complete Exam-Ready Algorithm

GENERAL ALGORITHM (Exam Writing Format):

1. **START**
2. Import `train_test_split` from `sklearn.model_selection`, `GaussianNB` from `sklearn.naive_bayes`, `metrics` from `sklearn`, and `load_iris` from `sklearn.datasets`.
3. Load the benchmark Iris dataset into memory using `iris = load_iris()`.
4. Assign the continuous feature measurements to array x via `iris.data` and ground-truth species target labels to array y via `iris.target`.
5. Partition the data into training (70%) and testing (30%) subsets with a fixed reproducible seed: `x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=40)`.
6. Instantiate the Gaussian Naive Bayes classifier: `gnb = GaussianNB()`.
7. Fit the classifier on the training data using `gnb.fit(x_train, y_train)`, which estimates the mean (μ) and variance (σ^2) for each feature per class.
8. Predict target class labels for the unseen test features: `y_pred = gnb.predict(x_test)`.
9. Calculate the classification accuracy score: `acc = metrics.accuracy_score(y_test, y_pred)`.
10. Generate a comprehensive classification report: `report = metrics.classification_report(y_test, y_pred, target_names=iris.target_names)`.
11. Print the accuracy score and the classification report to the console.
12. **STOP**

Section 4: Complete Theory

1. Bayes' Theorem in Machine Learning

Given a vector of continuous features $X = (x_1, x_2, x_3, x_4)$, we wish to find the class $y \in \{0, 1, 2\}$ that maximizes the posterior probability:

$$P(y|X) = rac{P(X|y) \cdot P(y)}{P(X)}$$

- **Prior Probability $P(y)$:** The baseline probability of observing class y before looking at any measurements (in Iris, each species has equal prior probability: $P(y) = 50/150 = 1/3$).
- **Likelihood $P(X|y)$:** The probability of observing measurements X given that the sample belongs to class y.
- **Evidence $P(X)$:** The total probability of observing measurements X across all classes (acts as a constant normalizer).
- **Posterior Probability $P(y|X)$:** The updated probability that a sample belongs to class y given its observed feature values.

2. The "Naive" Assumption of Conditional Independence

Calculating the full joint likelihood $P(x_1, x_2, x_3, x_4 | y)$ would require millions of samples. Naive Bayes simplifies this by assuming that all features are conditionally independent given class y:

$$P(x_1, x_2, x_3, x_4 | y) = P(x_1|y) imes P(x_2|y) imes P(x_3|y) imes P(x_4|y) = \prod_{i=1}^{4} P(x_i|y)$$
The predicted class is simply the class with the highest product:

$$\hat{y} = rg\max_{y} \left(P(y) \prod_{i=1}^{n} P(x_i|y) ight)$$

3. Gaussian Naive Bayes for Continuous Measurements

Because flower measurements (sepal length, petal width) are continuous real numbers, Gaussian Naive Bayes assumes each feature follows a normal (Gaussian) distribution within each class:

$$P(x_i | y) = rac{1}{\sqrt{2\pi \sigma_y^2}} \exp\left(- rac{(x_i - \mu_y)^2}{2\sigma_y^2} ight)$$
During training (`gnb.fit(x_train, y_train)`), the algorithm computes only two parameters for each feature per class: the mean (μ) and variance (σ^2). This makes training extremely fast ($O(N \cdot d)$) with tiny memory requirements.

4. Worked Bayes' Theorem Example

Suppose we have 2 classes (A and B) and 1 test measurement $x_1 = 3.0$:

- Priors: $P(A) = 0.5$, $P(B) = 0.5$.
- Class A parameters: $\mu_A = 2.0, \sigma_A^2 = 1.0$. Likelihood: $P(x=3|A) = rac{1}{\sqrt{2\pi(1)}} e^{-(3-2)^2 / 2} = 0.3989 imes e^{-0.5} pprox 0.242$.
- Class B parameters: $\mu_B = 5.0, \sigma_B^2 = 1.0$. Likelihood: $P(x=3|B) = rac{1}{\sqrt{2\pi(1)}} e^{-(3-5)^2 / 2} = 0.3989 imes e^{-2.0} pprox 0.054$.
- Posterior A numerator: $0.242 imes 0.5 = \mathbf{0.121}$.
- Posterior B numerator: $0.054 imes 0.5 = \mathbf{0.027}$.
- Since $0.121 > 0.027$, the sample is classified as **Class A**!

5. Naive Bayes vs. K-Nearest Neighbors

Dimension	Gaussian Naive Bayes (e06)	K-Nearest Neighbors (e05)
Model Type	Probabilistic generative model	Instance-based, lazy distance learner
Training Phase	Eager (computes μ and σ^2 per feature)	Lazy (stores training data in memory)
Inference Speed	Extremely fast ($O(C \cdot d)$ evaluations)	Slow on large datasets ($O(N \cdot d)$ distance calculations)
Feature Interaction	Assumes features are conditionally independent	Considers multi-dimensional geometric distances

Section 5: Complete Original Source Code & Walkthrough

Complete Verbatim Source Code (`e06_naive_bayes.py`)

```
# Exercise 6: Naive Bayes

# Question 1: Program to implement Naive Bayes Algorithm using any standard dataset available in the public domain and find the accuracy of the
algorithm.

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics
from sklearn.datasets import load_iris

iris = load_iris()

x = iris.data
y = iris.target

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=40
)

gnb = GaussianNB()

gnb.fit(x_train, y_train)

y_pred = gnb.predict(x_test)

acc = metrics.accuracy_score(y_test, y_pred)

report = metrics.classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
)

print(acc)
print(report)
```

Block-by-Block Execution Walkthrough

Block 1: Imports and Dataset Loading (Lines 5–13)

- `from sklearn.model_selection import train_test_split`: Imports array partitioning function.
- `from sklearn.naive_bayes import GaussianNB`: Imports Gaussian Naive Bayes estimator.
- `from sklearn import metrics`: Imports accuracy and classification report tools.
- `from sklearn.datasets import load_iris`: Imports Iris dataset.
- `iris = load_iris()`: Loads dataset Bunch.
- `x = iris.data`: Matrix of shape $(150, 4)$ containing measurements.
- `y = iris.target`: Array of shape $(150,)$ containing species labels $0, 1, 2$.

Block 2: Reproducible Train/Test Partition (Lines 15–17)

- `train_test_split(x, y, test_size=0.3, random_state=40)`:
  - `test_size=0.3`: Allocates 30% of data (45 samples) to testing and 70% (105 samples) to training.
  - `random_state=40`: Fixes the pseudo-random seed, guaranteeing identical sample selection on every execution.

Block 3: Model Initialization, Training, and Prediction (Lines 19–23)

- `gnb = GaussianNB()`: Instantiates the classifier.
- `gnb.fit(x_train, y_train)`: Fits the model to 105 training samples, calculating mean ($\mu_{c, j}$) and standard deviation ($\sigma_{c, j}$) for each of the 4 features across all 3 classes (only 24 numbers learned!).
- `y_pred = gnb.predict(x_test)`: Computes posterior probabilities for the 45 test flowers and assigns the species with maximum probability.

Block 4: Evaluation Metrics & Display (Lines 25–34)

- `acc = metrics.accuracy_score(y_test, y_pred)`: Compares predicted array `y_pred` to true array `y_test`. Under seed 40, all 45 samples match perfectly $\implies 1.0$ (100%).
- `report = metrics.classification_report(..., target_names=iris.target_names)`: Builds precision, recall, and F1-score summary for Setosa, Versicolor, and Virginica.
- `print(acc)` and `print(report)`: Outputs results to console.

Section 6: Complete Function, Parameter and Syntax Reference

Source Code Coverage Checklist

Item in Source	Type	Library / Module	Verification Status
`load_iris()`	Function	`sklearn.datasets`	Confirmed present in source (Line 9)
`iris.data` / `iris.target`	Attributes	Scikit-Learn Bunch	Confirmed present in source (Lines 11-12)
`iris.target_names`	Attribute	Scikit-Learn Bunch	Confirmed present in source (Line 30)
`train_test_split()`	Function	`sklearn.model_selection`	Confirmed present in source (Line 15)
`test_size=0.3`	Parameter	`train_test_split`	Confirmed present in source (Line 16)
`random_state=40`	Parameter	`train_test_split`	Confirmed present in source (Line 16)
`GaussianNB()`	Class	`sklearn.naive_bayes`	Confirmed present in source (Line 19)
`.fit(x_train, y_train)`	Method	Classifier Estimator	Confirmed present in source (Line 21)
`.predict(x_test)`	Method	Classifier Estimator	Confirmed present in source (Line 23)
`metrics.accuracy_score()`	Function	`sklearn.metrics`	Confirmed present in source (Line 25)
`metrics.classification_report()`	Function	`sklearn.metrics`	Confirmed present in source (Line 27)
`target_names=`	Parameter	`classification_report`	Confirmed present in source (Line 30)

Detailed 10-Point Breakdown for Key Functions

1. **`GaussianNB()`**
   1. **Exact Name:** `sklearn.naive_bayes.GaussianNB`
   2. **Module:** `sklearn.naive_bayes`
   3. **Purpose:** Implements the Gaussian Naive Bayes algorithm for continuous classification.
   4. **General Syntax:** `model = GaussianNB(priors=None, var_smoothing=1e-9)`
   5. **Parameters:** `priors`: Prior probabilities of the classes (if None, calculated from data); `var_smoothing`: Portion of largest variance added to variances for numerical stability.
   6. **Small Example:** `clf = GaussianNB().fit([[1, 2], [3, 4]], [0, 1])`
   7. **Expected Result:** An eager parametric probabilistic model.
   8. **Role in Program:** Core classifier trained on `x_train` and evaluated on `x_test`.
   9. **Common Mistakes:** Capitalization mistakes: typing `gaussianNB` or `GaussianNb`.
   10. **Memory Tip:** "Gaussian Normal Bell: Capital G, capital NB."

2. **`metrics.classification_report()`**
   1. **Exact Name:** `sklearn.metrics.classification_report`
   2. **Module:** `sklearn.metrics`
   3. **Purpose:** Builds a formatted text report showing main classification metrics per class.
   4. **General Syntax:** `classification_report(y_true, y_pred, target_names=None)`
   5. **Parameters:** `y_true`: True labels; `y_pred`: Predicted labels; `target_names`: Display labels matching class indices.
   6. **Small Example:** `print(classification_report([0, 1], [0, 1], target_names=['Cat', 'Dog']))`
   7. **Expected Result:** Tabular string containing precision, recall, f1-score, and support columns.
   8. **Role in Program:** Evaluates detailed multi-class performance on the 45 test flowers.
   9. **Common Mistakes:** Omitting `target_names=iris.target_names` (shows 0, 1, 2 instead of species names).
   10. **Memory Tip:** "Classification report gives Precision, Recall, F1, and Support."

Section 7: Step-by-Step Execution Walkthrough

Step	Line	Action	Array Dimensions & State
1	9-12	Extract Iris arrays	`x` : $(150, 4)$ floats; `y` : $(150,)$ integers
2	15-17	Partition data with seed 40	`x_train` : $(105, 4)$, `y_train` : $(105,)$ `x_test` : $(45, 4)$, `y_test` : $(45,)$
3	19	Instantiate GaussianNB	Model object initialized with default parameters
4	21	Fit training data	Estimates 3 class priors ($1/3$), 12 means (μ), and 12 variances (σ^2)
5	23	Predict test set	`y_pred` shape: $(45,)$ predicted classes
6	25	Compute accuracy	Compares `y_test` with `y_pred` $\implies 45/45 = \mathbf{1.0}$
7	27-31	Build report	Tabulates precision, recall, and f1 for Setosa, Versicolor, Virginica
8	33-34	Display results	Prints `1.0` followed by the full classification table

Section 8: Output and Interpretation

Complete Console Output (100% Deterministic)

```
1.0
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        14
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        18

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Metric-by-Metric Interpretation

- **Accuracy (1.0):** Indicates 100% accuracy; every single one of the 45 test flowers was classified correctly.
- **Support Column:** Shows the class distribution in the test partition:
  - Setosa: 14 test flowers.
  - Versicolor: 13 test flowers.
  - Virginica: 18 test flowers.
  - Total test flowers: $14 + 13 + 18 = 45$.
- **Precision (1.00):** Whenever the model predicted Setosa, Versicolor, or Virginica, it was correct 100% of the time ($FP = 0$).
- **Recall (1.00):** The model successfully identified 100% of all real Setosa, Versicolor, and Virginica flowers in the test set ($FN = 0$).
- **F1-Score (1.00):** The harmonic mean of Precision and Recall is at its theoretical maximum of 1.00.
- **Macro vs. Weighted Average:** Macro averages treat each class equally ($1/3$ each); weighted averages weight each class by its support ($14/45, 13/45, 18/45$). Here, both are 1.00.

Section 9: Exam Preparation

A. Practical-Record Answer Format

- **Aim:** To implement the Gaussian Naive Bayes classification algorithm using the standard Iris dataset and evaluate model performance using accuracy score and classification report.
- **Algorithm:** Reproduce the 11-step algorithm from Section 3.
- **Key Code Statements:**

```
iris = load_iris(); x, y = iris.data, iris.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=40)
gnb = GaussianNB().fit(x_train, y_train)
y_pred = gnb.predict(x_test)
print(metrics.accuracy_score(y_test, y_pred))
print(metrics.classification_report(y_test, y_pred, target_names=iris.target_names))
```

- **Result:** Gaussian Naive Bayes classifier achieved 100% accuracy (1.00) on the Iris test set with precision, recall, and F1-scores of 1.00 across all 3 flower species.

B. Question Bank (2, 3, and 5 Marks)

- **2 Marks:** State Bayes' Theorem and write its mathematical formula.
- **3 Marks:** What is the difference between Precision and Recall in a classification report?
- **5 Marks:** Explain Gaussian Naive Bayes. Why is it called "naive", and how does it handle continuous feature measurements?

Section 10: Viva Voce Questions and Answers (16 Detailed Q&As)

Viva Q1: What is Bayes' Theorem?

Answer: Bayes' Theorem calculates the posterior probability of a class y given observed features X: $P(y|X) = ↑rac{P(X|y) P(y)}{P(X)}$.

Viva Q2: Why is the Naive Bayes algorithm called "naive"?

Answer: Because it makes the strong, naive assumption that all features are conditionally independent of each other given the class label.

Viva Q3: When is GaussianNB specifically used compared to MultinomialNB or BernoulliNB?

Answer: GaussianNB is used when features are continuous real numbers following a normal distribution. MultinomialNB is used for discrete counts (e.g. text word counts), and BernoulliNB is used for binary boolean features.

Viva Q4: What does GaussianNB learn during the training phase (.fit())?

Answer: It calculates and stores only two parameters for each feature per class: the mean (μ) and the variance (σ^2), alongside class prior probabilities.

Viva Q5: Why is random_state=40 included in train_test_split?

Answer: It fixes the random seed so that the pseudo-random data split is identical on every execution, guaranteeing reproducible results.

Viva Q6: What is the formula for Precision?

Answer: $ ext{Precision} = ↑rac{TP}{TP + FP}$ (True Positives divided by total predicted positives).

Viva Q7: What is the formula for Recall?

Answer: $ ext{Recall} = ↑rac{TP}{TP + FN}$ (True Positives divided by total actual positives).

Viva Q8: What is the F1-Score?

Answer: It is the harmonic mean of Precision and Recall: $F_1 = 2 imes ↑rac{ ext{Precision} imes ext{Recall}}{ ext{Precision} + ext{Recall}}$.

Viva Q9: What does the Support column in the classification report mean?

Answer: Support is the actual number of occurrences of each true class in the test dataset (14 Setosa, 13 Versicolor, 18 Virginica).

Viva Q10: What is the difference between Macro Average and Weighted Average?

Answer: Macro average computes the simple unweighted mean across classes. Weighted average weights the metric of each class by its support in the dataset.

Viva Q11: What is the training time complexity of Naive Bayes?

Answer: It is $O(N \cdot d)$, where N is the number of samples and d is the number of features, making it linear and one of the fastest classifiers in machine learning.

Viva Q12: What is the "Zero Frequency Problem" in Naive Bayes and how is it solved?

Answer: If a categorical feature value never occurs with a class in the training data, its probability is 0, wiping out the entire product. It is solved using Laplace smoothing (adding 1 to counts).

Viva Q13: Does Naive Bayes require feature scaling like normalization or standardization?

Answer: No. GaussianNB estimates mean and variance separately for each feature, so differences in scale between features do not distort the probabilities.

Viva Q14: Is Gaussian Naive Bayes sensitive to irrelevant features?

Answer: Yes. Because probabilities are multiplied across all features, many noisy or correlated features can degrade classification performance.

Viva Q15: How many test samples are evaluated in e06_naive_bayes.py?

Answer: 45 samples (30% of the 150 Iris samples).

Viva Q16: Why does e06 achieve 1.0 (100%) accuracy?

Answer: Because the Iris dataset is small, clean, and well-separated in feature space, and the seed `random_state=40` creates a particularly clean train/test split.

Section 11: Common Mistakes and Troubleshooting

Common Mistake	Error Message / Symptom	Correction
Misspelling `GaussianNB`	`ImportError: cannot import name 'gaussianNB'`	Ensure capital `G` and capital `NB`: `from sklearn.naive_bayes import GaussianNB`.
Swapping arguments in `accuracy_score(y_pred, y_test)`	Gives correct value for accuracy, but crashes on directional metrics	Always pass ground-truth first: `accuracy_score(y_test, y_pred)`.
Omitting `target_names=iris.target_names`	Report prints numerical rows `0, 1, 2` instead of flower names	Pass `target_names=iris.target_names` inside `classification_report()`.
Omitting `random_state=40`	Accuracy varies between 0.93 and 1.00 instead of exact 1.00	Include `random_state=40` inside `train_test_split()` for reproducible marks.

Section 12: Memory Tricks & Last-Minute Rapid Revision

> **Master Mnemonic for Naive Bayes Code Flow:**
> **"L-S-G-F-P-A-R"**
> **Load Iris $ ightarrow$ Split Train/Test (seed 40) $ ightarrow$ GaussianNB Model $ ightarrow$ Fit Training Data $ ightarrow$ Predict Test Data $ ightarrow$ Accuracy Score $ ightarrow$ Report (Classification Report)**

LAST-MINUTE EXAM SKELETON:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics

iris = load_iris()
x, y = iris.data, iris.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=40)

gnb = GaussianNB().fit(x_train, y_train)
y_pred = gnb.predict(x_test)

print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
print(metrics.classification_report(y_test, y_pred, target_names=iris.target_names))
```

Top 5 High-Priority Viva Questions (Last-Minute Refresh)

1. What is the fundamental assumption of the Naive Bayes algorithm?
Answer: The "naive" assumption of class-conditional feature independence: all features are mutually independent given the class label.
2. Why use GaussianNB instead of MultinomialNB on the Iris dataset?
Answer: Because Iris features are continuous real numbers assumed to follow a normal (Gaussian) distribution, whereas MultinomialNB is for discrete count data.
3. What does Bayes' theorem state?
Answer: $P(C_k \mid \mathbf{x}) = rac{P(C_k) \cdot P(\mathbf{x} \mid C_k)}{P(\mathbf{x})}$, relating posterior probability to prior, likelihood, and evidence.
4. What is the difference between Precision and Recall?
Answer: Precision is $rac{ ext{TP}}{ ext{TP}+ ext{FP}}$ (exactness / purity of positive predictions); Recall is $rac{ ext{TP}}{ ext{TP}+ ext{FN}}$ (completeness / sensitivity in finding all actual positives).
5. Why is `random_state=40` specified in `train_test_split()`?
Answer: To seed the pseudo-random number generator deterministically, ensuring reproducible train/test subsets and identical test metrics across executions.

13. Cross-Program Comparison Table (Programs 05 to 10)

The table below provides a rigorous cross-program comparative reference across all machine learning programs in the laboratory curriculum, highlighting learning paradigms, dataset characteristics, data splitting strategies, core algorithms, evaluation metrics, and diagnostic visualizations:

Program	Learning Paradigm	Dataset Used	Data Partitioning	Core Algorithm & Parameters	Evaluation Metrics	Diagnostic Visualizations
e05_knn.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.25` (75:25 split)	K-Nearest Neighbors (`n_neighbors=5`)	`accuracy_score()`	2 Subplots: Predicted vs Actual classification on test samples
e06_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3, random_state=40` (70:30)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score()` & `classification_report()`	None (Structured terminal classification report)
e07_decision_tree.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3` (70:30 split)	Decision Tree (`criterion="entropy"`, `max_depth=4`, `min_samples_leaf=2`)	`classification_report()`	Full visual flowchart tree via `tree.plot_tree(filled=True)`
e08_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`train_size=0.8` (80:20 split)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score() * 100` (%)	None (Live interactive console input parsing with `float` & species mapping)
e09_kmeans.py	**Unsupervised Clustering**	Synthetic Blobs (300 × 2)	None (Full unlabeled dataset)	K-Means Clustering (`n_clusters=5`, `random_state=42`, `n_init="auto"`)	Inertia / Cluster assignments (`labels_`)	2D Scatter plot with `c=labels, cmap="viridis"` and centroids marked with red 'x'
e10_multiple_linear_regression.py	**Supervised Regression**	Diabetes Dataset (Features 0 & 2)	Slicing (`[:-20]` train, `[-20:]` test)	Multiple Ordinary Least Squares (`LinearRegression()`)	Mean Squared Error (MSE) & R^2 Score	Figure 1: Feature vs Target subplots; Figure 2: Actual vs Predicted fit line